PROGRAMMING & CS FUNDAMENTALS

Arrays
Visual Guide

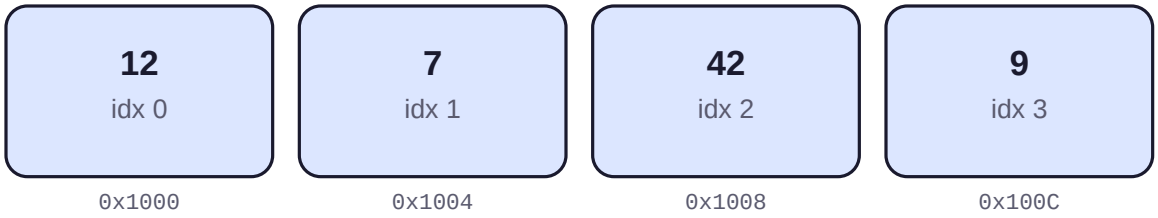
Contiguous memory, O(1) indexing —
the foundation every other structure is built on.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Contiguous Memory Layout

Elements sit side by side; the index IS the address math.




```
address(i) = base + i * elem_size
arr[2] -> 0x1000 + 2*4 = 0x1008   # one multiply-add: O(1)
```

- No pointers between elements — the CPU computes locations arithmetically.
- Fixed size at allocation: the block must be contiguous, so growing means moving.
- elem_size is known at compile time (int=4B, double=8B) — hence the simple math.

STEP 2/6

Operation Costs

What each operation really pays.

Read arr[i] — O(1)

address math

Append (space) — O(1)*

write at end

Insert at k — O(n)

shift k..n right

Delete at k — O(n)

shift k+1..n left

Search (unsorted) — O(n)

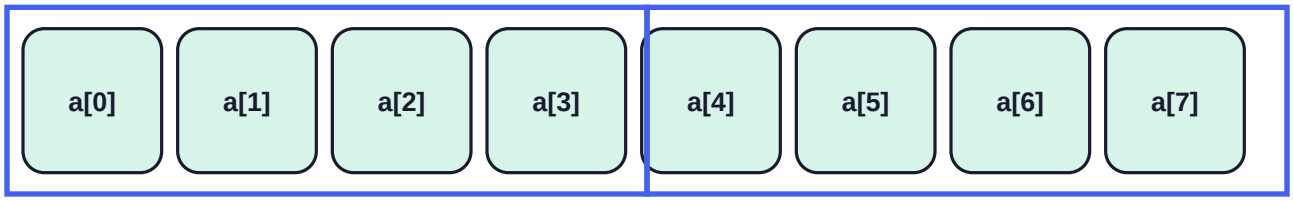
scan

*Append is O(1) amortized for dynamic arrays (see step 4) — but O(n) worst case when a resize fires.

STEP 3/6

Cache Locality: The Hidden Superpower

Sequential access rides the CPU cache; random access stalls it.



Reading a[0] pulls the whole 64-byte line into L1 — a[1..15] are then nearly free.

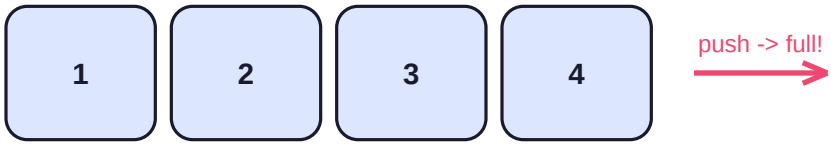
- Spatial locality: neighbors arrive together — linear scans are 10-100x faster than pointer hops.
- Why arrays beat linked lists in practice despite identical Big O for scans.
- Strided access (every 16th element) defeats the cache — same O(n), far slower wall-clock.

STEP 4/6

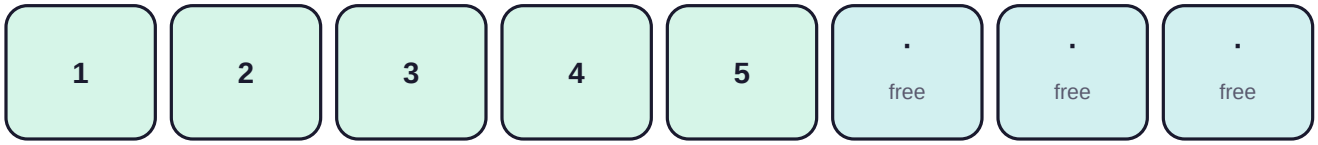
Dynamic Arrays: Growth Strategy

Capacity doubles when full — copies are rare by design.

Before push (cap=4, size=4):



After: allocate 8, copy 4, then push:




```
total copy cost for n pushes: 1+2+4+...+n/2 < n
=> O(1) amortized per push      # doubling is the trick
```

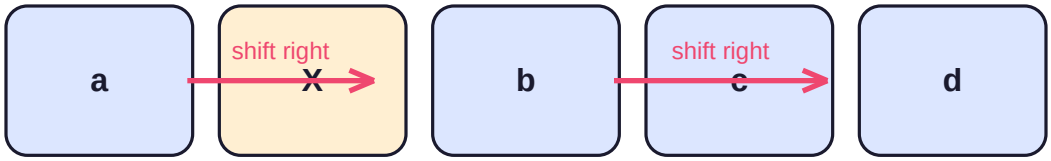
Grow by 1.5x (not 2x) if memory is tight — fewer wasted slots, slightly more copies. Python lists over-allocate ~1.125x.

STEP 5/6

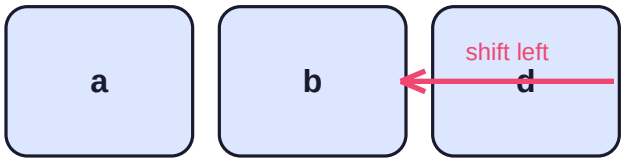
Insert & Delete: The Shift Tax

Everything after the hole moves — that's the O(n).

Insert X at index 1:



Delete at index 2:



- Insert/delete at the END is O(1); at the FRONT is O(n) — design around this.
- Swap-with-last trick: unordered delete in O(1) by overwriting with the tail.
- Frequent middle inserts? That's the linked list's home turf.

STEP 6/6

Array Pitfalls

Small mistakes, big crashes.

- Off-by-one: valid indices are 0..n-1. The #1 bug in array code — draw the fenceposts.
- Bounds are unchecked in C: arr[100] on a 10-array reads garbage or segfaults.
- Stack overflow: a 10MB array as a local variable blows the typical 8MB stack — malloc it.
- 0-based vs 1-based: know your language; mixing them corrupts index math.
- ArrayList vs LinkedList: default to ArrayList — cache locality wins until proven otherwise.
- Multidimensional arrays are still flat: a[i][j] = base + (i*cols + j) * size.

STEP ✓I✓

Key Takeaways

Arrays — Remember

- Contiguous memory + index math (base + i*size) gives O(1) random access.
- Sequential scans exploit cache lines — arrays beat pointer structures in wall-clock time.
- Dynamic arrays double capacity on fill: O(1) amortized append, O(n) worst-case resize.
- Insert/delete in the middle costs O(n) shifts; swap-with-last gives O(1) unordered delete.
- Watch off-by-ones, unchecked bounds, and stack allocation of big arrays.

Keep exploring

Pair this guide with the matching video walkthroughs
and hands-on labs at interstitiumlabs.dev